The Journal of Open Source Software

OpenSimula: a component-based time simulation environment in Python for buildings and HVAC systems

Juan F. Coronel [1¶], **Luis Pérez-Lombard** [1], **and Ismael R. Maestre** [2]

1 Departamento de Ingeniería Energética, Escuela Técnica Superior de Ingeniería, Universidad de Sevilla, Spain **2** Departamento de Máquinas y Motores Térmicos, Escuela Politécnica Superior de Algeciras, Universidad de Cádiz, Spain ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 29 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

OpenSimula is an open-source Python package for the dynamic simulation of buildings and their heating, ventilation and air-conditioning (HVAC) systems. A simulation is a set of components (weather files, schedules, materials and constructions, spaces and surfaces, HVAC equipment and systems) that reference each other and are advanced together in time steps, typically hourly over a year. Models are defined as Python dictionaries or JSON files, run from a script or a Jupyter or marimo notebook, and their results are returned as NumPy arrays and pandas DataFrames (Harris et al., 2020; McKinney, 2010), so they can be analysed, plotted and post-processed with the scientific Python ecosystem without intermediate files. The package includes the input definitions of the ANSI/ASHRAE Standard 140-2023 test cases (*ANSI/ASHRAE Standard 140-2023*, 2023) and a script that runs all of them and compares the results with those of the reference programs published with the Standard.

Statement of need

Building energy simulation is used to design buildings and HVAC systems, check them against energy codes, and research new envelope technologies and control strategies. The established tools, such as EnergyPlus (Crawley et al., 2001), TRNSYS (Klein & others, 2017) and ESP-r (Clarke, 2001), are standalone programs: a model is written in a program-specific input format, the simulation runs as an external process, and the results are read back from output files. Parametric studies, optimisation, sensitivity analysis or the coupling with data-driven and machine learning models therefore require an additional layer that writes input files, launches runs and parses outputs.

OpenSimula targets researchers, teachers and students who work in Python and want the building and HVAC models themselves to be Python objects. Every parameter and every result of every component is accessible in memory between and after simulations; a parametric study is a loop that changes parameter values and calls `simulate()` again; and a new physical model is a Python class. Because the source code of the models is short and readable, the package is also suited to teaching how a building simulation program works internally, from conduction transfer functions to the iterative coupling of zones and HVAC systems. The component architecture is not specific to buildings, so other time-varying thermal systems, such as refrigeration equipment or industrial processes, can be modelled with the same framework (Coronel et al., 2026).

DRAFT

State of the field

EnergyPlus, TRNSYS and ESP-r are the reference programs of the Standard 140 comparisons and the most widely used whole-building simulation engines; OpenStudio (Guglielmetti et al., 2011) and Python libraries such as eppy (Philip, 2024) make EnergyPlus scriptable, but the model still runs as an external executable whose internal state is not accessible during the simulation. The Modelica Buildings library (Wetter et al., 2014) offers equation-based, fully extensible component models, at the cost of a Modelica toolchain and a modelling paradigm that is far from the day-to-day tools of most Python users. Within the Python ecosystem there are well-established packages for parts of the problem, such as PsychroLib for psychrometrics (Meyer & Thevenard, 2019), which OpenSimula uses, or pvlib for solar irradiance (Holmgren et al., 2018), but no package that covers the whole chain from weather data and envelope to air-side and water-side HVAC systems and that has been checked against the full set of Standard 140 test suites. OpenSimula aims to fill this gap with a pure Python implementation whose models are deliberately close to the ones used by the reference programs, so that its results can be compared with them case by case.

Software design

OpenSimula has three levels. A `Simulation` contains one or more `Projects`; a project holds the simulation parameters (time step, number of steps, initial date, weather file, shadow calculation mode) and a list of `Components`. Each component has typed *parameters*, which define it and are validated before the simulation (ranges, allowed options, references to other components of the same or another project), and *variables*, the time series it produces. Components are coupled by references (a space references its space type and building, a surface its construction and space, an HVAC system its spaces, equipment and schedules), and by `Parameter_math_exp` expressions that can use variables of other components, which allows control laws such as a setpoint that depends on the outdoor temperature to be written in the project definition without new code.

During the simulation the project calls the components in a fixed order of types (schedules, surfaces, openings, spaces, buildings, HVAC systems, calculators) and, within each time step, repeats their `iteration()` method until all of them report convergence. Coupled quantities, such as the air temperature and humidity of a zone conditioned by an HVAC system, are solved by fixed-point iteration with an adaptive relaxation coefficient. The available models include, among others: weather files in MET, TMY2, TMY3 and WYEC2 formats with isotropic, Hay–Davies, Reindl and Perez (Perez et al., 1990) sky models; multilayer constructions solved with conduction transfer functions; exterior, interior, ground-coupled and virtual surfaces; windows with angle-dependent glazing properties; external shading computed geometrically with Shapely (Gillies & others, 2025) on a 3D model of the building, which can be displayed with Plotly (Plotly Technologies Inc., 2015); interior long-wave radiation between surfaces by radiosity (Coronel, 2012); ideal loads systems; direct-expansion units and systems; and single-zone and multi-zone water-based air systems with coils, fans, pumps, chillers and heat pumps. Psychrometric properties are computed with PsychroLib (Meyer & Thevenard, 2019). New component types are added by subclassing `Component`, declaring parameters and variables and implementing the simulation methods; the API reference documents these base classes.

The design favours readability and accessibility of every intermediate value over speed: the core is plain Python and NumPy, and an annual hourly simulation of a single-zone Standard 140 case takes of the order of ten seconds on a laptop.

DRAFT

Validation

The `ASHRAE_140` folder of the repository contains the definitions of the test cases of Standard 140-2023 (*ANSI/ASHRAE Standard 140-2023*, 2023; Judkoff & Neymark, 1995) as project JSON files: weather drivers (WD100–WD600), building thermal envelope and fabric loads (cases 195–995 and free-floating cases), ground-coupled slab-on-grade (GC30b–GC80c), space cooling equipment (CE100–CE545), fuel-fired furnaces (HE100–HE230) and air-side HVAC equipment (AE101–AE445). The script `validate.py` simulates all of them in parallel, extracts the outputs requested by the Standard and compares them with the range (minimum to maximum) of the example results of the reference programs; it also generates the validation page of the documentation. The same comparison runs in continuous integration on every change to the code, so that any change in the results of the 135 test projects is detected.

With version 0.8.9, 2,379 of the 2,740 single-valued outputs (87 %) lie inside the range of the reference programs, and almost half of the remaining ones are within 1 % of the nearest bound. Figure 1 shows the annual heating and cooling loads of the building envelope cases. The cases outside the range and their probable causes, such as the constant interior convection coefficients that produce a small, systematic overestimation of heating loads, are documented in the repository. The range of the reference programs is not the Standard's acceptance criterion, but it shows that the results of OpenSimula are of the same quality as those of established simulation programs.

DRAFT

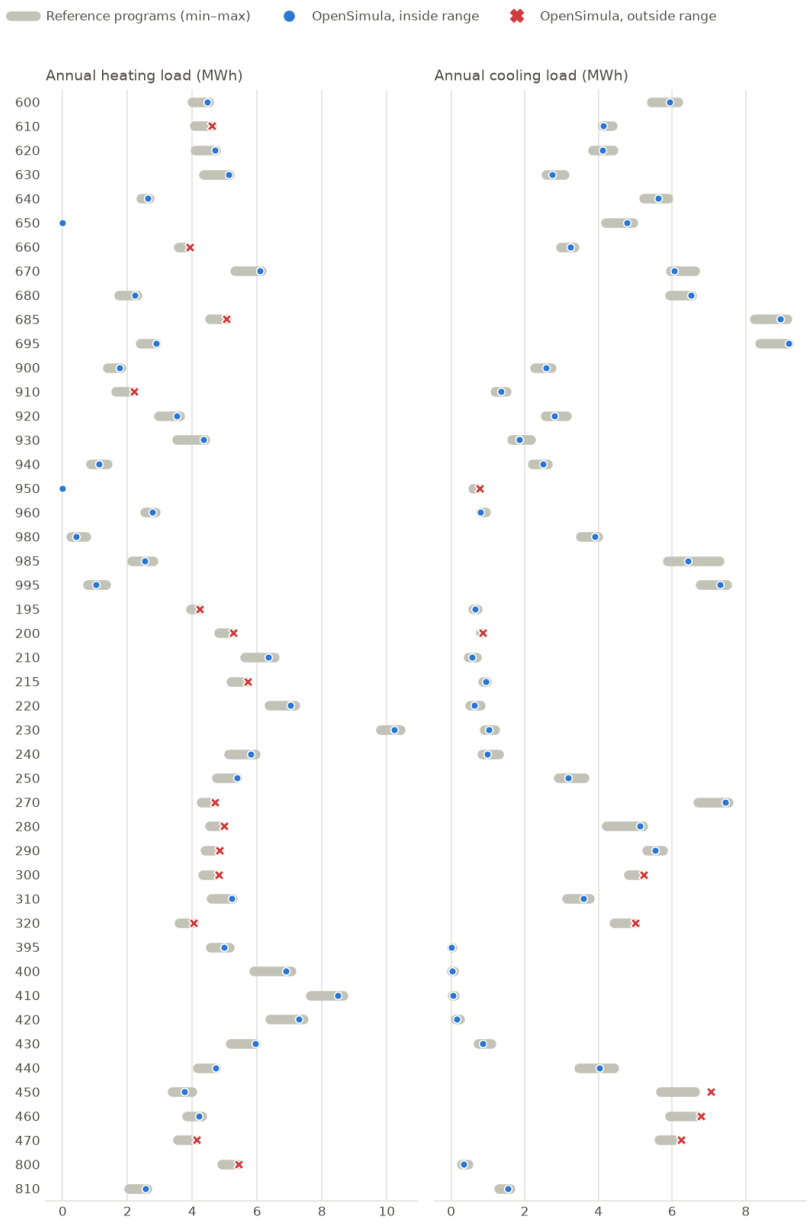


Figure 1: Annual heating and cooling loads of the Standard 140 building envelope cases: range of the reference programs (grey) and OpenSimula (dots inside the range, crosses outside).

Research impact statement

OpenSimula has been developed at the Universidad de Sevilla since 2023, in collaboration with the Universidad de Cádiz, and is distributed on PyPI, with each release archived on Zenodo. It has been presented at the CYTEF 2026 congress (Coronel et al., 2026).

More than three bachelor's and master's theses at the Universidad de Sevilla have used OpenSimula as their simulation tool, and more are in progress. One of them, close to completion, imports building models from HULC, the official Spanish software for the energy performance assessment of buildings under the national building code, so that OpenSimula can simulate the buildings already modelled by practitioners for compliance; a second one, planned, will import EnergyPlus IDF models. OpenSimula is also being introduced in master's

and doctoral courses on building energy simulation, where students can read and modify the models they use.

AI usage disclosure

Generative AI tools (Claude Code, Anthropic) were used during the development of recent versions, mainly to automate the Standard 140 validation workflow, write tests and documentation, and investigate and fix discrepancies found by the validation, and to prepare a first draft of this paper. All changes were reviewed by the authors, who take full responsibility for the software and the paper.

Author contributions

J. F. Coronel conceived OpenSimula, designed its architecture, wrote most of the code and is its lead developer and maintainer. L. Pérez-Lombard reviewed and advised on the HVAC equipment and system models, and I. R. Maestre on the building envelope models. Both have been the main testers of the software, helped to define the many validation test cases and contributed to the writing of this paper.

Acknowledgements

The development of OpenSimula has not received specific funding from any project or grant.

References

ANSI/ASHRAE standard 140-2023: Method of test for evaluating building performance simulation software. (2023). American Society of Heating, Refrigerating; Air-Conditioning Engineers.

Clarke, J. A. (2001). *Energy simulation in building design* (2nd ed.). Butterworth-Heinemann.

Coronel, J. F. (2012). *Simulación térmica de entornos arquitectónicos: Aproximación a los espacios exteriores* [PhD thesis, Universidad de Sevilla]. http://hdl.handle.net/11441/15303

Coronel, J. F., Pérez-Lombard, L., & Maestre, I. R. (2026). OpenSimula: Entorno de simulación energética basado en componentes en Python. *CYTEF 2026, XIII Congreso Ibérico y XI Congreso Iberoamericano de Ciencias y Técnicas Del Frío*.

Crawley, D. B., Lawrie, L. K., Winkelmann, F. C., Buhl, W. F., Huang, Y. J., Pedersen, C. O., Strand, R. K., Liesen, R. J., Fisher, D. E., Witte, M. J., & Glazer, J. (2001). EnergyPlus: Creating a new-generation building energy simulation program. *Energy and Buildings*, *33*(4), 319–331. https://doi.org/10.1016/S0378-7788(00)00114-6

Gillies, S., & others. (2025). *Shapely: Manipulation and analysis of geometric objects*. https://github.com/shapely/shapely.

Guglielmetti, R., Macumber, D., & Long, N. (2011). OpenStudio: An open source integrated analysis platform. *Proceedings of Building Simulation 2011: 12th Conference of the International Building Performance Simulation Association*, 442–449.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Holmgren, W. F., Hansen, C. W., & Mikofski, M. A. (2018). Pvlib python: A python package for modeling solar energy systems. *Journal of Open Source Software*, *3*(29), 884.

DRAFT

https://doi.org/10.21105/joss.00884

Judkoff, R., & Neymark, J. (1995). *International energy agency building energy simulation test (BESTEST) and diagnostic method* (NREL/TP-472-6231). National Renewable Energy Laboratory.

Klein, S. A., & others. (2017). *TRNSYS 18: A transient system simulation program*. Solar Energy Laboratory, University of Wisconsin-Madison.

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

Meyer, D., & Thevenard, D. (2019). PsychroLib: A library of psychrometric functions to calculate thermodynamic properties of air. *Journal of Open Source Software*, *4*(33), 1137. https://doi.org/10.21105/joss.01137

Perez, R., Ineichen, P., Seals, R., Michalsky, J., & Stewart, R. (1990). Modeling daylight availability and irradiance components from direct and global irradiance. *Solar Energy*, *44*(5), 271–289. https://doi.org/10.1016/0038-092X(90)90055-H

Philip, S. (2024). *Eppy: Scripting language for E+ idf files and E+ output files*. https://github.com/santoshphilip/eppy.

Plotly Technologies Inc. (2015). *Collaborative data science*. https://plot.ly.

Wetter, M., Zuo, W., Nouidui, T. S., & Pang, X. (2014). Modelica Buildings library. *Journal of Building Performance Simulation*, *7*(4), 253–270. https://doi.org/10.1080/19401493.2013.765506

DRAFT